

Curso desarrollo de aplicaciones con tecnologías web

UF1842. Desarrollo y reutilización de componentes software y multimedia mediante lenguajes de guión.

Arquitectura de aplicaciones web

La arquitectura web es el diseño y estructura organizativa de una aplicación o sistema web. Define cómo interactúan sus componentes principales (cliente, servidor, bases de datos y red) para proporcionar servicios a los usuarios finales.

En esencia, la arquitectura web es el “esqueleto” que conecta y coordina diferentes elementos tecnológicos para garantizar que una aplicación sea funcional, escalable, segura y eficiente.

Esquema General de Aplicaciones Web

Arquitectura de Capas

Interacción entre Capas Cliente y Servidor

Arquitectura de la Capa Cliente

Esquema General de Aplicaciones Web

Teoría

Una aplicación web es un programa accesible a través de un navegador, que utiliza tecnologías web para proporcionar funcionalidad interactiva.

- **Cliente:** Generalmente un navegador web. Interpreta el código HTML, CSS y JavaScript para mostrar la interfaz de usuario
- **Servidor:** Procesa las solicitudes del cliente, ejecuta la lógica de negocio y devuelve las respuestas, generalmente en forma de HTML, JSON o XML.
- **Base de datos:** Almacena y gestiona la información que necesita la aplicación.
- **Red:** Infraestructura que conecta cliente y servidor, permitiendo la transmisión de datos mediante protocolos como HTTP/HTTPS.

Ejemplo Práctico

1. Un usuario abre el navegador e ingresa a una URL.
2. El navegador (cliente) envía una solicitud al servidor mediante HTTP.
3. El servidor procesa la solicitud, consulta la base de datos si es necesario, y devuelve una respuesta.
4. El navegador interpreta la respuesta y la presenta al usuario.

Ejercicio

Crea un diagrama en una herramienta como **FigJam**, **LucidChart** o **a mano alzada** que ilustre el flujo de datos en una aplicación web.

Arquitectura de Capas

Teoría

La arquitectura de capas organiza las aplicaciones en diferentes niveles, donde cada capa tiene una responsabilidad específica:

1. **Capa de presentación:** Interactúa con el usuario, mostrando la interfaz gráfica.
 - Ejemplo: Una página web con un formulario de contacto.
2. **Capa de lógica de negocio:** Procesa datos y ejecuta reglas de negocio.
 - Ejemplo: Validar los datos del formulario en el servidor.
3. **Capa de datos:** Gestiona la información almacenada en bases de datos.
 - Ejemplo: Guardar un mensaje de contacto en una base de datos MySQL.

Ejemplo Práctico

Diseñar una aplicación sencilla con estas capas:

- **Frontend:** Formulario HTML para capturar datos del usuario.
- **Backend:** Script PHP que recibe los datos y los valida.
- **Base de datos:** Tabla MySQL para almacenar los datos enviados.

Interacción entre Capas Cliente y Servidor

Teoría

La interacción entre cliente y servidor sigue el modelo solicitud-respuesta:

1. El cliente realiza una solicitud HTTP.
2. El servidor procesa la solicitud y devuelve una respuesta.

Las solicitudes pueden ser:

- **Síncronas:** Bloquean la ejecución hasta recibir una respuesta.
- **Asíncronas:** Permiten continuar la ejecución mientras esperan la respuesta.

Tecnologías clave:

- **AJAX** y **fetch API** para solicitudes asíncronas.
- **JSON** como formato de intercambio de datos.

Ejemplo Práctico

Crea una página web que permita buscar información en una API pública, como la API de OpenWeatherMap, para mostrar el clima en tiempo real.

Arquitectura de la Capa Cliente

Teoría

La capa cliente utiliza tecnologías como:

- **HTML** para la estructura.
- **CSS** para el diseño.
- **JavaScript** para la interacción.

Además, los frameworks como **React**, **Vue.js** y **Angular** simplifican el desarrollo de aplicaciones dinámicas.

Tecnologías clave:

- **AJAX** y **fetch API** para solicitudes asíncronas.
- **JSON** como formato de intercambio de datos.

Ejemplo Práctico

Crea una aplicación en **Angular** que permita mostrar una lista de productos y filtrarlos por categoría.